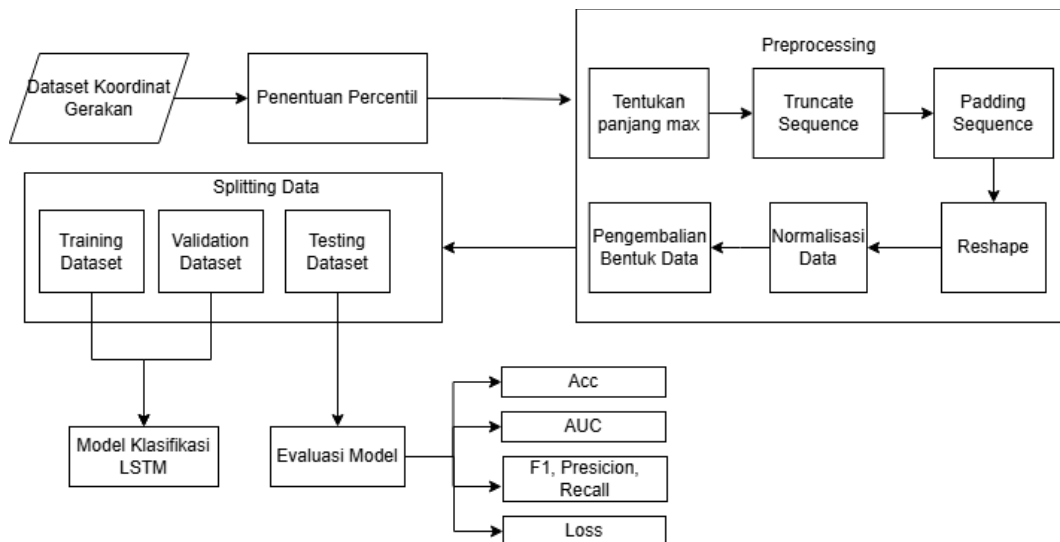


3. METODE PENELITIAN

3.1 Tahapan Penelitian

Metode penelitian yang digunakan pada penelitian ini bertujuan untuk membangun model klasifikasi gerakan rehabilitasi menggunakan algoritma *Long Short-Term Memory* (LSTM) berdasarkan data koordinat gerakan tubuh yang diperoleh dari dataset UI-PRMD. Secara garis besar tahapan penelitian yang dilakukan dimulai dari pengolahan dataset koordinat gerakan hingga evaluasi performa model klasifikasi. Tahapan-tahapan penelitian tersebut dapat dilihat pada Gambar 3.1.



Gambar 3. 1 Metode Penelitian Klasifikasi Gerakan

Tahapan pertama yang dilakukan adalah mengumpulkan dan mempersiapkan dataset koordinat gerakan rehabilitasi yang diperoleh dari dataset UI-PRMD. Dataset yang diperoleh kemudian dianalisis untuk menentukan nilai persentil yang digunakan sebagai dasar dalam menentukan panjang maksimum *sequence* yang akan digunakan pada proses *preprocessing*. Tahap *preprocessing* dilakukan melalui beberapa langkah, yaitu menentukan panjang maksimum *sequence*, melakukan *truncate sequence* pada data yang melebihi panjang maksimum, serta melakukan *padding sequence* pada data yang memiliki panjang lebih pendek. Setelah seluruh *sequence* memiliki panjang yang sama, data diubah bentuknya agar sesuai dengan

format input yang dibutuhkan oleh model LSTM. Selanjutnya dilakukan normalisasi data untuk menyamakan rentang nilai antar fitur sehingga proses pelatihan model dapat berjalan lebih optimal. Setelah proses normalisasi, data dikembalikan ke bentuk yang sesuai. Dataset dibagi menjadi *training* dan *testing* dataset. *Training* dataset digunakan untuk melatih model klasifikasi LSTM agar mampu mempelajari pola gerakan rehabilitasi yang terdapat pada data. *Testing* dataset digunakan untuk menguji kemampuan model dalam melakukan klasifikasi terhadap data yang belum pernah digunakan selama proses pelatihan. Tahap selanjutnya adalah pembangunan model klasifikasi menggunakan algoritma *Long Short-Term Memory* (LSTM). Model dilatih menggunakan *training* dataset untuk mempelajari hubungan temporal antar koordinat gerakan tubuh dan menghasilkan prediksi kelas gerakan yang sesuai. Setelah proses pelatihan selesai, dilakukan evaluasi model menggunakan *testing* dataset. Evaluasi dilakukan dengan menggunakan beberapa metrik, yaitu *Accuracy* (Acc), *Area Under Curve* (AUC), *Precision*, *Recall*, *F1-Score*, dan *Loss*. Hasil evaluasi akan digunakan untuk mengukur tingkat performa model dalam mengklasifikasikan gerakan rehabilitasi serta menentukan efektivitas metode yang diusulkan.

3.2. Pengumpulan Dataset UI-PRMD

Metode pertama yang dilakukan dalam penelitian ini adalah mengumpulkan dataset yang digunakan sebagai objek penelitian dalam proses klasifikasi kualitas gerakan rehabilitasi. Dataset yang digunakan adalah University of Idaho – *Physical Rehabilitation Movement Dataset* (UI-PRMD), yaitu dataset publik yang berisi rekaman gerakan rehabilitasi dalam bentuk koordinat tubuh (*skeletal joint positions*) yang direkam menggunakan sensor Vicon dan Kinect untuk 10 subjek dan 10 jenis gerakan rehabilitasi, masing-masing dengan label gerakan benar (*correct*) dan gerakan salah (*incorrect*). Dataset UI-PRMD pada awalnya tersedia melalui laman resmi University of Idaho. Namun pada saat proses pengumpulan data dilakukan, laman resmi tersebut tidak dapat diakses secara langsung, sehingga pengumpulan data dilakukan melalui repository GitHub yang menyediakan berkas

dataset UI-PRMD, yaitu melalui tautan berikut: <https://github.com/niemasd/UI-PRMD-Analysis/tree/master/data>.

Setiap file dalam dataset berbentuk *file* .txt/.csv yang berisi kumpulan *frame*, di mana setiap baris merepresentasikan satu *frame* waktu dan setiap kolom merepresentasikan koordinat (x, y, z) dari satu *joint* tubuh. Contoh struktur potongan data ditampilkan pada Tabel 3.1.

Tabel 3.1 Contoh Struktur Data Koordinat Gerakan UI-PRMD

Frame	Joint1_X	Joint1_Y	Joint1_Z	Joint2_X	Joint2_Y	Joint2_Z	...	Label
1	0.1023	0.5871	-0.2214	0.1187	0.5920	-0.2050	...	<i>Correct</i>
2	0.1030	0.5865	-0.2198	0.1192	0.5911	-0.2033	...	<i>Correct</i>
3	0.1041	0.5850	-0.2170	0.1200	0.5895	-0.2010	...	<i>Correct</i>
...	...	...	...	...	...	...	...	...
150	0.1305	0.5620	-0.1890	0.1420	0.5710	-0.1720	...	<i>Incorrect</i>

Seperti dapat dilihat pada Tabel 3.1, jumlah kolom *joint* disesuaikan dengan jumlah titik tubuh yang direkam pada masing-masing jenis gerakan (*movement type*), dan jumlah frame berbeda-beda tergantung durasi gerakan. Setiap gerakan memiliki jumlah frame yang bervariasi, data ini bersifat *sequential* (berurutan waktu), sehingga sesuai dengan kebutuhan arsitektur *Long Short-Term Memory* (LSTM) yang memang dirancang untuk mempelajari pola dari data berbentuk *time-series* atau *sequence*. Secara lengkap, dataset yang digunakan pada penelitian ini dapat di lihat pada Lampiran 2.

Proses pengumpulan data pada penelitian ini dilakukan melalui tahapan yang dirangkum dalam Algoritma 3.1 :

Algoritma 3.1. Proses Pengumpulan Data UI-PRMD	
Input	URL repository GitHub yang menyediakan dataset UI-PRMD (https://github.com/niemasd/UI-PRMD-Analysis/tree/master/data)
Proses	1. Mengakses laman resmi University of Idaho untuk mengunduh dataset UI-PRMD.

2. Jika laman resmi tidak dapat diakses, maka beralih menggunakan repository GitHub sebagai sumber alternatif.
3. Mengunduh seluruh file dataset gerakan rehabilitasi dari repository GitHub tersebut.
4. Mengekstrak file dataset yang telah diunduh.
5. Memeriksa struktur folder dataset (jenis gerakan, subjek, label *correct/incorrect*).
6. Memeriksa format setiap file (.txt/.csv).
7. Membaca isi file dan memvalidasi kelengkapan data koordinat (x, y, z) pada setiap frame.
8. Menyimpan file yang valid ke direktori dataset lokal; menandai file yang rusak/tidak lengkap (jika ada).

Output

Kumpulan file dataset gerakan rehabilitasi (*raw dataset*) yang telah diverifikasi dan siap digunakan pada tahap pre-processing.

Proses pengumpulan data pada penelitian ini diimplementasikan melalui fungsi *load_dataset()* yang bertugas membaca seluruh file dataset UI-PRMD dari direktori lokal, baik data gerakan dengan label *correct* maupun *incorrect*. Berikut adalah potongan *pseudocode* dari proses pengumpulan data tersebut.

```
def load_dataset(base_dir, sensor='Kinect',
                signal_type='Angles'):
    X, y, meta = [], [], []

    # Load Correct Movements (label=1)
    correct_dir = os.path.join(base_dir, 'Segmented Movements',
                                sensor, signal_type)
    correct_files = sorted(glob.glob(os.path.join(correct_dir,
                                                    '*.gz')))
    print(f"Ditemukan {len(correct_files)} file CORRECT di:
    {correct_dir}")

    for file in correct_files:
        data = load_gz_file(file)
        if data is not None and len(data) > 0:
            fname = os.path.basename(file)
            match = re.match(r'm(\d+)_s(\d+)_e(\d+)', fname)
            mov, subj, ep = match.groups() if match else ('?',
            '?', '?')
            X.append(data)
            y.append(1)
            meta.append({'file': fname, 'label': 'correct',
            'label_id': 1,
                        'movement': int(mov), 'subject':
            int(subj), 'episode': int(ep)})

    incorrect_dir = os.path.join(base_dir, 'Incorrect Segmented
    Movements', sensor, signal_type)
```

```

        incorrect_files =
sorted(glob.glob(os.path.join(incorrect_dir, '*.gz')))
        print(f"Ditemukan {len(incorrect_files)} file INCORRECT di:
{incorrect_dir}")

        for file in incorrect_files:
            data = load_gz_file(file)
            if data is not None and len(data) > 0:
                fname = os.path.basename(file)
                match = re.match(r'm(\d+)_s(\d+)_e(\d+)', fname)
                mov, subj, ep = match.groups() if match else ('?',
'?', '?')
                X.append(data)
                y.append(0)
                meta.append({'file': fname, 'label': 'incorrect',
'label_id': 0,
'movement': int(mov), 'subject':
int(subj), 'episode': int(ep)})

        return X, np.array(y), pd.DataFrame(meta)

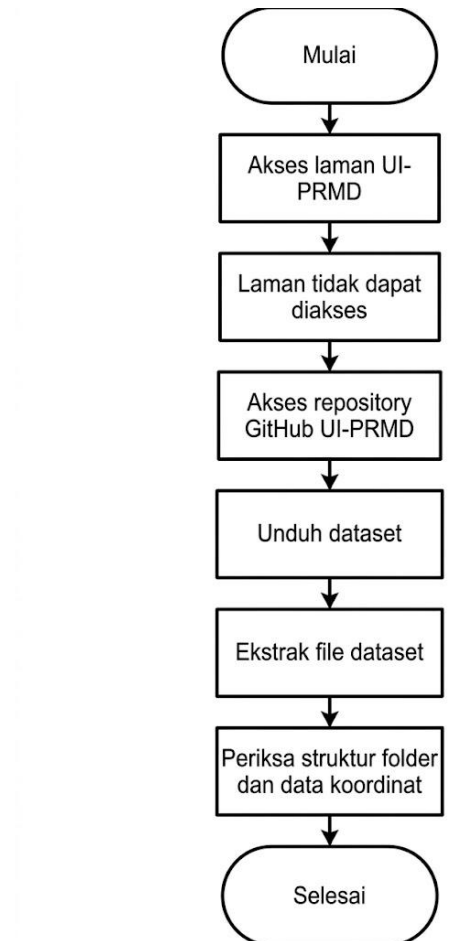
BASE_DIR = 'data'
SENSOR = 'Kinect'
SIGNAL_TYPE = 'Angles'

X_raw, y, meta_df = load_dataset(BASE_DIR, sensor=SENSOR,
signal_type=SIGNAL_TYPE)

```

Fungsi *load_dataset()* bekerja dengan membaca data gerakan dari dua direktori berbeda, yaitu folder *Segmented Movements* yang berisi data gerakan benar (*correct*) dan folder *Incorrect Segmented Movements* yang berisi data gerakan salah (*incorrect*), dengan parameter *sensor* dan *signal_type* digunakan untuk menentukan jenis sensor (*Kinect*) dan jenis sinyal (*Angles*) yang akan diambil datanya. Setiap file dataset berformat *.gz* dibaca menggunakan fungsi *load_gz_file()*, kemudian nama file diekstraksi menggunakan *regular expression* untuk memperoleh informasi identitas gerakan (*movement*), subjek (*subject*), dan episode (*episode*) dari data tersebut. Data yang berhasil dibaca kemudian disimpan ke dalam list *X* sebagai data koordinat gerakan, list *y* sebagai label kelas (1 untuk *correct* dan 0 untuk *incorrect*), serta list meta sebagai informasi metadata setiap file. Pada akhir proses, fungsi ini mengembalikan data *X_raw* yang digunakan sebagai data mentah untuk tahap *pre-processing*, label *y* dalam bentuk *array*, serta *meta_df* dalam bentuk *dataframe* yang berisi rincian metadata dari seluruh data yang telah

dikumpulkan. Proses pengumpulan data UI-PRMD dapat dilihat pada *Flowchart* Gambar 3.2.



Gambar 3.2. *Flowchart* Proses Pengumpulan Data UI-PRMD

3.3. Penentuan Persentil

Penentuan persentil merupakan tahapan yang dilakukan untuk mendapatkan acuan panjang *sequence* (jumlah *frame*) yang akan digunakan pada seluruh data gerakan sebelum masuk ke tahap *pre-processing*. Hal ini diperlukan karena setiap data gerakan pada dataset UI-PRMD memiliki jumlah *frame* yang berbeda-beda, sedangkan model LSTM membutuhkan panjang *sequence* yang seragam (*fixed-length input*) agar dapat diproses dalam bentuk *batch* saat pelatihan. Sebagai ilustrasi, variasi panjang *sequence* dari beberapa data gerakan ditampilkan pada Tabel 3.2.

Tabel 3.2 Contoh Variasi Panjang *Sequence* Data Gerakan

No	Nama File Gerakan	Jenis Gerakan	Jumlah Frame
1	m01_s01_e01.txt	Deep Squat	142
2	m01_s01_e02.txt	Deep Squat	158
3	m02_s01_e01.txt	Hurdle Step	121
4	m03_s02_e01.txt	Inline Lunge	175

Dalam menentukan panjang *sequence* acuan, digunakan pendekatan persentil ke-95 (P95). Nilai P95 dipilih karena mampu merepresentasikan panjang *sequence* yang mencakup 95% dari keseluruhan data tanpa terlalu terpengaruh oleh *outlier*, yaitu data dengan jumlah *frame* yang jauh lebih panjang dibanding mayoritas data lainnya. Jika digunakan nilai maksimum, panjang *sequence* akan sangat dipengaruhi oleh *outlier* sehingga menyebabkan terlalu banyak *padding* pada data lain, sedangkan jika digunakan nilai rata-rata, banyak data akan kehilangan informasi penting akibat *truncation* yang berlebihan. Proses penentuan persentil menggunakan Algoritma 3.2.

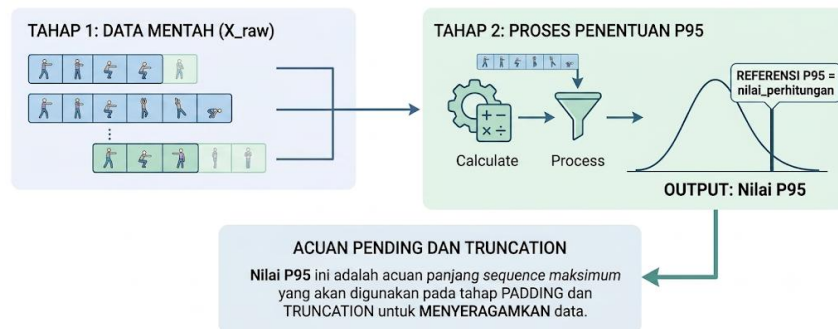
Algoritma 3.2. Proses Penentuan Persentil Panjang Sequence	
Input	X_{raw} (kumpulan data gerakan hasil pengumpulan data pada Tahap 3.2, berupa <i>array/list</i> , dengan setiap elemen berbentuk matriks berukuran [<i>jumlah frame</i> , <i>jumlah fitur</i>])
Proses	1. Menyiapkan variabel <i>lengths</i> sebagai penampung panjang <i>sequence</i> dari seluruh data. 2. Melakukan iterasi terhadap setiap data gerakan x pada X_{raw}. 3. Mengambil jumlah frame dari setiap data gerakan melalui $x.shape[0]$. 4. Menyimpan jumlah frame tersebut ke dalam variabel <i>lengths</i>. 5. Menghitung nilai persentil ke-95 dari seluruh nilai pada <i>lengths</i> menggunakan fungsi <code>np.percentile(lengths, 95)</code>. 6. Menampilkan nilai persentil ke-95 yang telah dihitung.
Output	Nilai persentil ke-95 (P95) yang digunakan sebagai acuan panjang maksimum <i>sequence</i> pada tahap <i>padding/truncation</i> di <i>preprocessing</i> .

Implementasi dari algoritma tersebut ke dalam *syntax* Python ditunjukkan sebagai berikut:

```
lengths = [x.shape[0] for x in X_raw]
print(f"P95      : {np.percentile(lengths, 95):.1f}")
```

Variabel *lengths* digunakan untuk menyimpan panjang *sequence* (jumlah *frame*) dari seluruh data gerakan pada *X_raw*, yang diperoleh dari atribut *x.shape[0]* setiap data. Selanjutnya, fungsi *np.percentile(lengths, 95)* digunakan untuk menghitung nilai persentil ke-95 dari seluruh panjang *sequence* tersebut. Nilai P95 tersebut selanjutnya digunakan sebagai acuan panjang *sequence* pada tahap *pre-processing*: data dengan jumlah *frame* lebih pendek dari nilai P95 akan dilakukan *padding*, sedangkan data dengan jumlah *frame* lebih panjang akan dilakukan *truncation*, sehingga seluruh data memiliki panjang *sequence* yang seragam sebelum digunakan sebagai *input* pada model LSTM.

Sebagai gambaran yang lebih jelas mengenai tahapan penentuan persentil, proses tersebut diilustrasikan pada Gambar 3.3, yang menunjukkan alur mulai dari data mentah (*X_raw*) dengan panjang *sequence* yang bervariasi hingga diperoleh nilai persentil ke-95 (P95) sebagai acuan panjang *sequence* maksimum. Nilai P95 tersebut selanjutnya digunakan sebagai dasar pada tahap *padding* dan *truncation* untuk menyeragamkan panjang seluruh data sebelum digunakan sebagai *input* model LSTM.



Gambar 3.3. Ilustrasi Proses Penentuan Persentil Panjang *Sequence*